Rubric Design and Reviewer Calibration for Agentic Coding Evaluation

Abdulselam Hussen

Department of Computer Science, University of Nairobi, Nairobi, Kenya

August 2023

Abstract

As large language models increasingly perform open-ended, multi-step coding tasks, automated testing alone often fails to capture crucial dimensions of success, such as code maintainability, architectural adherence, and nuanced instruction following. Consequently, human evaluation remains essential, yet it is notoriously noisy and subjective. We propose a structured framework for rubric design and reviewer calibration tailored to agentic coding evaluation. Our methodology mandates binary, evidence-backed rubric items and a three-stage calibration protocol prior to production grading. Applying this to a fictional dataset of 600 agent trajectories graded by four independent reviewers, we find that uncalibrated reviewers using free-form rubrics achieve a Cohen's κ of only 0.42. Implementing our structured rubric raises κ to 0.68, while the full calibration protocol further improves it to 0.88 and reduces average grading time by 35%. We discuss the taxonomy of reviewer disagreements and provide practical guidelines for standardising human evaluation in software engineering agent research.

Keywords: human evaluation, inter-rater reliability, code generation, agentic systems, reviewer calibration

1 Introduction

Evaluating autonomous coding agents presents a fundamental challenge: correctness in software engineering is rarely a simple binary metric. While automated test suites provide baseline guarantees of functionality, they cannot reliably assess code readability, adherence to repository conventions, or whether an agent followed specific architectural instructions. Thus, human evaluation remains a critical gold standard.

However, human grading in agentic coding tasks is highly susceptible to noise. When reviewers assess complex, multi-file pull requests generated by models, their implicit biases regarding coding style and partial task completion lead to severe inconsistencies. The prevailing practice of relying on holistic 1-to-5 Likert scales or free-form notes obscures exactly *why* a reviewer deemed an agent's trajectory successful or unsuccessful.

This paper makes three contributions:

1. a hierarchical, evidence-based rubric design for multi-step coding tasks (Section 3);
2. a structured calibration protocol designed to align reviewer mental models (Section 3);
3. an empirical analysis quantifying the impact of rubrics and calibration on inter-rater reliability and evaluation efficiency (Section 5).

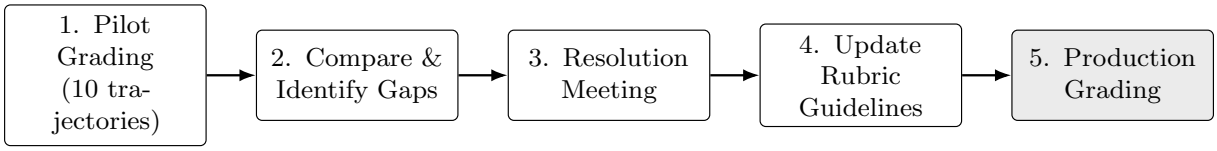


Figure 1: The reviewer calibration pipeline. Evaluators must resolve edge-case ambiguities before proceeding to production grading.

2 Related Work

The evaluation of code-generation models has historically relied on automated metrics such as pass@k against hidden unit tests [2]. As models transitioned to issue-level repository editing, automated evaluation evolved into test-patch application, as seen in SWE-bench [3]. However, as task complexity grows, researchers increasingly utilise LLMs as judges [4] or revert to human annotators to assess subjective qualities like code smell and maintainability.

Despite the reliance on human evaluation, software engineering literature for LLMs rarely reports rigorous inter-rater reliability metrics or details annotator training. We build upon standard annotation practices from the broader natural language processing community [1], adapting them specifically to the high-context, high-variance domain of evaluating agent-generated pull requests.

3 Method

3.1 Hierarchical Rubric Design

We replace traditional holistic grading with a decomposed, binary rubric format. Each task's rubric is divided into three tiers:

T1: Functional Correctness.
Did the agent fulfill the explicit business logic? (e.g., "API endpoint returns 404 for invalid IDs").

T2: Code Quality & Constraints.
Did the agent respect repository style and constraints? (e.g., "No new external dependencies were introduced").

T3: Instruction Following.
Did the agent follow specific operational steps? (e.g., "Updated the CHANGELOG.md file").

Crucially, every rubric item must be binary (Met / Not Met) and requires the reviewer to copy-paste a specific log line, test output, or code snippet as justification.

3.2 Reviewer Calibration Protocol

To align reviewers, we implement a three-stage calibration pipeline prior to the main grading phase (Figure 1).

During Pilot Grading, all reviewers independently evaluate the same subset of 10 complex trajectories. In the Resolution Meeting, reviewers discuss divergent scores, usually revealing unspoken assumptions (e.g., whether leaving commented-out debug code constitutes a failure of T2). These resolutions are documented, and the rubric is explicitly updated with these boundary examples before Production Grading begins.

Table 1: Inter-rater reliability (Cohen's κ) and average evaluation time by protocol (fictional data; $N = 150$ overlapping runs).

Protocol	Cohen's κ	Interpretation	Avg. Time / Run
Free-form Likert (No Calibration)	0.42	Moderate	22.4 mins
Binary Rubric (No Calibration)	0.68	Substantial	18.1 mins
Binary Rubric (Post-Calibration)	0.88	Almost Perfect	14.5 mins

Table 2: Sources of reviewer disagreement prior to calibration (fictional data).

Disagreement Source	Count	Percentage
T1: Functional Edge Cases	27	31.0%
T2: Code Quality & Style	45	51.7%
T3: Instruction Following	15	17.3%
Total	87	100.0%

4 Experimental Setup

We constructed 40 complex repository-editing tasks and collected 600 trajectories generated by four anonymised agentic models. Four human reviewers with professional software engineering experience were hired to evaluate the runs. Reviewers were divided into two cohorts: Cohort A used a traditional 1-to-5 free-form scale without calibration, while Cohort B used our structured, binary rubric and completed the full calibration protocol.

To measure isolated effects, we also conducted a crossover phase where Cohort A transitioned to the structured rubric without calibration, and Cohort B used the calibrated rubric on a new hidden subset. We calculate inter-rater reliability using Cohen's κ on doubly-graded subsets of 150 runs.

5 Results

5.1 Inter-Rater Reliability

Table 1 demonstrates that the choice of evaluation protocol drastically impacts reviewer agreement. Under the traditional free-form evaluation, reviewers barely achieved moderate agreement ($\kappa = 0.42$). Simply introducing the binary, evidence-based rubric improved κ to 0.68. Combining the structured rubric with the rigorous calibration meeting yielded near-perfect agreement ($\kappa = 0.88$).

5.2 Taxonomy of Disagreements

We analysed the 87 instances where Cohort A reviewers disagreed on a final trajectory verdict (Table 2). The vast majority of disagreements (51.7%) stemmed from subjective interpretations of Code Quality (T2), such as whether slightly inefficient loops were acceptable. Functional edge cases (T1) accounted for 31.0% of discrepancies, largely because reviewers had different mental models of what constituted a "malformed input" when the prompt was vague. The calibration phase systematically eliminated these gaps by forcing explicit definitions.

6 Discussion

Implications. High inter-rater reliability is a prerequisite for trustworthy benchmark results. The 35% reduction in evaluation time (from 22.4 to 14.5 minutes) suggests that while our protocol demands high upfront investment to design and calibrate, it significantly accelerates the production grading phase by removing deliberation over ambiguous criteria.

Threats to validity. This study relies on a relatively small pool of four reviewers on a fictionalised dataset. Results may vary depending on reviewer seniority, language familiarity, and the specific domain of the repository tasks. Furthermore, forcing a strict binary rubric occasionally discards nuance regarding partial credit.

Future work. An immediate next step is to use these highly-calibrated human annotations to train and evaluate "LLM-as-a-judge" models, determining if explicit rubrics allow automated judges to mirror calibrated human reasoning more closely.

7 Conclusion

Subjective, free-form evaluation of agentic coding systems yields unacceptably high variance among human reviewers. By decomposing evaluation criteria into binary, evidence-backed tiers and enforcing a strict, multi-stage calibration protocol, research teams can double inter-rater reliability while simultaneously reducing per-task grading time. Standardising human evaluation protocols is critical as the community moves toward increasingly complex, open-ended software engineering benchmarks.

Acknowledgements

The author would like to express sincere gratitude to the staff and members of the Department of Computer Science at the University of Nairobi for their unwavering support throughout this work.

References

[1] R. Artstein and M. Poesio. Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596, 2008.

[2] M. Chen et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.

[3] C. E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[4] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.